

1. Objective

To validate and test the deep learning prediction API built in Task 4 using industry-standard API testing tools - Postman and curl - covering both successful requests and deliberate failure cases, and to document the results in a structured test report.

2. Reference Tools and API Under Test

- **Postman** - a Postman Collection (`postman_collection.json`) was built covering every endpoint, importable directly into the Postman application, with built-in test assertions on status code and response structure for each request.
- **curl** - an equivalent shell script (`curl_commands.sh`) issues the same requests from the command line.
- **Flask API from Task 4** - the Digit Classification API (`app.py`), serving the Task 1 CNN model (97.04% test accuracy).
- **Automated Python test runner** - `test_api_validation.py` mirrors every request in the Postman collection programmatically (using the `requests` library) and asserts on status codes and response fields, producing a PASS/FAIL report equivalent to a Postman Collection Runner / `newman` run, for environments without the Postman GUI installed.

3. Approach

1. **Send prediction requests** - valid single-image and batch prediction requests are sent to `/predict` and `/predict/batch`.
2. **Validate JSON responses** - every response is checked to be valid JSON with the expected fields (`predicted_digit` , `confidence` , `probabilities` , or `error`), and correctness is checked directly against known true labels from the original digits dataset.
3. **Perform negative testing** - deliberately malformed requests are sent: a missing required field, a wrong-size input array, an invalid JSON body, out-of-range pixel values, an empty batch list, an unknown endpoint, and a request made with the wrong HTTP method.
4. **Verify error-handling mechanisms** - every negative test is checked for the correct HTTP status code (400/404/405) and a structured, human-readable JSON error body rather than a raw server stack trace.
5. **Document test results** - every test case, its expected outcome, and its actual (passed/failed) result are recorded in `API_TESTING_REPORT.md` and summarized below.

4. Source Code - Postman Collection (`postman_collection.json`)

The complete Postman Collection, importable directly into Postman, covering 10 requests across positive and negative test groups, each with built-in test assertions:

```
{
  "info": {
    "name": "Digit Classification API - Task 5 Testing",
    "description": "Postman collection for testing and validating the Task 4 Flask REST API (Digit Classification API). Covers positive (functional) tests and negative (error-handling) tests for every endpoint.",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "variable": [
    {
      "key": "base_url",
      "value": "http://127.0.0.1:5000"
    }
  ],
  "item": [
    {
      "name": "1. Positive Tests",
      "item": [
        {
          "name": "GET / - API Info",
          "request": {
            "method": "GET",
            "url": "{{base_url}}/"
          },
          "event": [
            {
              "listen": "test",
              "script": {
                "exec": [
                  "pm.test('Status code is 200', function () { pm.response.to.have.status(200); });",
                  "pm.test('Response has name and endpoints fields', function () {",
                  "  var json = pm.response.json();",
                  "  pm.expect(json).to.have.property('name');",
                  "  pm.expect(json).to.have.property('endpoints');",
                  "});"
                ]
              }
            }
          ]
        },
        {
          "name": "GET /health - Health Check",
          "request": {
```

```

    "method": "GET",
    "url": "{{base_url}}/health"
  },
  "event": [
    {
      "listen": "test",
      "script": {
        "exec": [
          "pm.test('Status code is 200', function () { pm.response.to.have.status(200); });",
          "pm.test('model_loaded is true', function () {",
            "  pm.expect(pm.response.json().model_loaded).to.eql(true);",
          "});"
        ]
      }
    }
  ]
},
{
  "name": "POST /predict - Valid Image (digit 0)",
  "request": {
    "method": "POST",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json"
      }
    ],
    "url": "{{base_url}}/predict",
    "body": {
      "mode": "raw",
      "raw": "{{\"image\": [0,0,5,13,9,1,0,0, 0,0,13,15,10,15,5,0, 0,3,15,2,0,11,8,0, 0,4,12,0,0,8,8,0, 0,5,8,0,0,9,8,0, 0,4,11,0,1,12,7,0, 0,2,14,5,10,12,0,0, 0,0,6,13,10,0,0,0]}}"
    }
  },
  "event": [
    {
      "listen": "test",
      "script": {
        "exec": [
          "pm.test('Status code is 200', function () { pm.response.to.have.status(200); });",
          "pm.test('Response has predicted_digit, confidence, probabilities', function () {",
            "  var json = pm.response.json();",
            "  pm.expect(json).to.have.property('predicted_digit');",
            "  pm.expect(json).to.have.property('confidence');",
            "  pm.expect(json).to.have.property('probabilities');",
          "});",
          "pm.test('predicted digit is 0 as expected', function () {",
            "  pm.expect(pm.response.json().predicted_digit).to.eql(0);",
          "});"
        ]
      }
    }
  ]
},
{
  "name": "POST /predict/batch - Valid Batch (3 real digits)",
  "request": {
    "method": "POST",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json"
      }
    ],
    "url": "{{base_url}}/predict/batch",
    "body": {
      "mode": "raw",
      "raw": "{{\"images\": [[0,0,5,13,9,1,0,0,0,0,13,15,10,15,5,0,0,3,15,2,0,11,8,0,0,4,12,0,0,8,8,0,0,5,8,0,0,9,8,0,0,4,11,0,1,12,7,0,0,2,14,5,10,12,0,0,0,0,6,13,10,0,0,0]]}}"
    }
  },
  "event": [
    {
      "listen": "test",
      "script": {
        "exec": [
          "pm.test('Status code is 200', function () { pm.response.to.have.status(200); });",
          "pm.test('Response has count and results', function () {",
            "  var json = pm.response.json();",
            "  pm.expect(json).to.have.property('count');",
            "  pm.expect(json).to.have.property('results');",
          "});"
        ]
      }
    }
  ]
},
{
  "name": "2. Negative Tests (Error Handling)",
  "item": [
    {

```

```

"name": "POST /predict - Missing 'image' field",
"request": {
  "method": "POST",
  "header": [
    {
      "key": "Content-Type",
      "value": "application/json"
    }
  ],
  "url": "{{base_url}}/predict",
  "body": {
    "mode": "raw",
    "raw": "{}"
  }
},
"event": [
  {
    "listen": "test",
    "script": {
      "exec": [
        "pm.test('Status code is 400', function () { pm.response.to.have.status(400); });",
        "pm.test('Response has error field', function () {",
        "  pm.expect(pm.response.json()).to.have.property('error');",
        "});"
      ]
    }
  }
]
},
{
  "name": "POST /predict - Wrong-size image array",
  "request": {
    "method": "POST",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json"
      }
    ],
    "url": "{{base_url}}/predict",
    "body": {
      "mode": "raw",
      "raw": "{\"image\": [1, 2, 3]}"
    }
  },
  "event": [
    {
      "listen": "test",
      "script": {
        "exec": [
          "pm.test('Status code is 400', function () { pm.response.to.have.status(400); });"
        ]
      }
    }
  ]
},
{
  "name": "POST /predict - Invalid JSON body",
  "request": {
    "method": "POST",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json"
      }
    ],
    "url": "{{base_url}}/predict",
    "body": {
      "mode": "raw",
      "raw": "not-json"
    }
  },
  "event": [
    {
      "listen": "test",
      "script": {
        "exec": [
          "pm.test('Status code is 400', function () { pm.response.to.have.status(400); });"
        ]
      }
    }
  ]
},
{
  "name": "POST /predict/batch - Empty images list",
  "request": {
    "method": "POST",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json"
      }
    ]
  },

```



```

print(f"[{status}] {name}" + (f" - {detail}" if detail else ""))

def check(name, condition, detail=""):
    record(name, bool(condition), detail)

SAMPLE_ZERO = [
    0, 0, 5, 13, 9, 1, 0, 0,
    0, 0, 13, 15, 10, 15, 5, 0,
    0, 3, 15, 2, 0, 11, 8, 0,
    0, 4, 12, 0, 0, 8, 8, 0,
    0, 5, 8, 0, 0, 9, 8, 0,
    0, 4, 11, 0, 1, 12, 7, 0,
    0, 2, 14, 5, 10, 12, 0, 0,
    0, 0, 6, 13, 10, 0, 0, 0,
]

def get_real_digit(digit):
    d = load_digits()
    idx = list(d.target).index(digit)
    return d.images[idx].flatten().tolist()

print("=" * 70)
print("SECTION 1: Positive (Functional) Tests")
print("=" * 70)

# --- GET / ---
r = requests.get(f"{BASE_URL}")
check("GET / returns 200", r.status_code == 200, f"status={r.status_code}")
body = r.json()
check("GET / response has 'name' and 'endpoints'",
      "name" in body and "endpoints" in body, json.dumps(body)[:80] + "...")

# --- GET /health ---
r = requests.get(f"{BASE_URL}/health")
check("GET /health returns 200", r.status_code == 200, f"status={r.status_code}")
check("GET /health reports model_loaded=true",
      r.json().get("model_loaded") is True, json.dumps(r.json()))

# --- POST /predict (valid) ---
r = requests.post(f"{BASE_URL}/predict", json={"image": SAMPLE_ZERO})
check("POST /predict (valid image) returns 200", r.status_code == 200, f"status={r.status_code}")
body = r.json()
check("POST /predict response has predicted_digit/confidence/probabilities",
      all(k in body for k in ("predicted_digit", "confidence", "probabilities")))
check("POST /predict correctly predicts digit 0",
      body.get("predicted_digit") == 0, f"predicted={body.get('predicted_digit')}, confidence={body.get('confidence')}")

# --- POST /predict/batch (valid, real digits 3, 7, 8) ---
images = [get_real_digit(3), get_real_digit(7), get_real_digit(8)]
r = requests.post(f"{BASE_URL}/predict/batch", json={"images": images})
check("POST /predict/batch (valid) returns 200", r.status_code == 200, f"status={r.status_code}")
body = r.json()
check("POST /predict/batch returns count == 3", body.get("count") == 3, json.dumps(body.get("count")))
preds = [item["predicted_digit"] for item in body.get("results", [])]
check("POST /predict/batch correctly predicts [3, 7, 8]", preds == [3, 7, 8], f"got {preds}")

print()
print("=" * 70)
print("SECTION 2: Negative Tests (Error Handling)")
print("=" * 70)

# --- Missing 'image' field ---
r = requests.post(f"{BASE_URL}/predict", json={})
check("POST /predict (missing image) returns 400", r.status_code == 400, f"status={r.status_code}")
check("POST /predict (missing image) response has 'error'", "error" in r.json(), json.dumps(r.json()))

# --- Wrong-size image array ---
r = requests.post(f"{BASE_URL}/predict", json={"image": [1, 2, 3]})
check("POST /predict (wrong size) returns 400", r.status_code == 400, f"status={r.status_code}")

# --- Invalid JSON body ---
r = requests.post(f"{BASE_URL}/predict", data="not-json", headers={"Content-Type": "application/json"})
check("POST /predict (invalid JSON) returns 400", r.status_code == 400, f"status={r.status_code}")

# --- Out-of-range pixel values ---
bad_pixels = [999] * 64
r = requests.post(f"{BASE_URL}/predict", json={"image": bad_pixels})
check("POST /predict (out-of-range pixels) returns 400", r.status_code == 400, f"status={r.status_code}")

# --- Empty batch list ---
r = requests.post(f"{BASE_URL}/predict/batch", json={"images": []})
check("POST /predict/batch (empty list) returns 400", r.status_code == 400, f"status={r.status_code}")

# --- Batch with one bad image among good ones (per-item isolation) ---
r = requests.post(f"{BASE_URL}/predict/batch", json={"images": [SAMPLE_ZERO, [1, 2, 3]]})
body = r.json()
check("POST /predict/batch (mixed valid/invalid) still returns 200", r.status_code == 200, f"status={r.status_code}")
check("POST /predict/batch isolates the bad image's error",
      "error" in body["results"][1] and "predicted_digit" in body["results"][0],

```

```

    json.dumps(body))

# --- Unknown endpoint ---
r = requests.get(f"{BASE_URL}/nonexistent")
check("GET /nonexistent returns 404", r.status_code == 404, f"status={r.status_code}")

# --- Wrong HTTP method ---
r = requests.get(f"{BASE_URL}/predict")
check("GET /predict (wrong method) returns 405", r.status_code == 405, f"status={r.status_code}")

# -----
print()
print("=" * 70)
print("TEST SUMMARY")
print("=" * 70)
total = len(results)
passed = sum(1 for r in results if r["passed"])
failed = total - passed
print(f"Total tests : {total}")
print(f"Passed      : {passed}")
print(f"Failed       : {failed}")
print(f"Pass rate    : {passed/total*100:.1f}%")

with open("test_results.json", "w") as f:
    json.dump({"total": total, "passed": passed, "failed": failed, "results": results}, f, indent=2)

if failed > 0:
    sys.exit(1)

```

6. Screenshots / Sample Outputs

6.1 Automated Test Run Output (Console Log)

The actual console output captured while running `test_api_validation.py` against the live Task 4 API:

```

=====
SECTION 1: Positive (Functional) Tests
=====
[PASS] GET / returns 200 - status=200
[PASS] GET / response has 'name' and 'endpoints' - {"description": "REST API serving the Task 1 from-scratch CNN digit classifier"...
[PASS] GET /health returns 200 - status=200
[PASS] GET /health reports model_loaded=true - {"model_loaded": true, "status": "ok"}
[PASS] POST /predict (valid image) returns 200 - status=200
[PASS] POST /predict response has predicted_digit/confidence/probabilities
[PASS] POST /predict correctly predicts digit 0 - predicted=0, confidence=0.9998
[PASS] POST /predict/batch (valid) returns 200 - status=200
[PASS] POST /predict/batch returns count == 3 - 3
[PASS] POST /predict/batch correctly predicts [3, 7, 8] - got [3, 7, 8]

=====
SECTION 2: Negative Tests (Error Handling)
=====
[PASS] POST /predict (missing image) returns 400 - status=400
[PASS] POST /predict (missing image) response has 'error' - {"error": "Missing 'image' field in request body."}
[PASS] POST /predict (wrong size) returns 400 - status=400
[PASS] POST /predict (invalid JSON) returns 400 - status=400
[PASS] POST /predict (out-of-range pixels) returns 400 - status=400
[PASS] POST /predict/batch (empty list) returns 400 - status=400
[PASS] POST /predict/batch (mixed valid/invalid) still returns 200 - status=200
[PASS] POST /predict/batch isolates the bad image's error - {"count": 2, "results": [{"confidence": 0.9998, "predicted_digit": 0,
"probabilities": {"0": 0.9998, "1": 0.0, "2": 0.0, "3": 0.0, "4": 0.0, "5": 0.0001, "6": 0.0, "7": 0.0, "8": 0.0, "9": 0.0}},
{"error": "image[1]: Expected an 8x8 (64-value) grayscale image, got 3 values."}]}
[PASS] GET /nonexistent returns 404 - status=404
[PASS] GET /predict (wrong method) returns 405 - status=405

=====
TEST SUMMARY
=====
Total tests : 20
Passed      : 20
Failed       : 0
Pass rate    : 100.0%

```

6.2 Equivalent curl Output (Console Log)

The actual console output captured while running `curl_commands.sh` against the same live API, confirming the same behavior via raw HTTP requests:

```
=== SECTION 1: Positive Tests ===

--- GET / ---
{"description":"REST API serving the Task 1 from-scratch CNN digit classifier.","endpoints":{"GET /health":"Health check.(","POST
/predict":"Predict the digit for one 8x8 grayscale image. Body: {\\"image\\": [64 numbers, 0-16 or 0-1, row-major 8x8]}","POST
/predict/batch":"Predict digits for multiple images. Body: {\\"images\\": [[64 numbers], [64 numbers], ...]}"},"name":"Digit
Classification API"}

HTTP Status: 200

--- GET /health ---
{"model_loaded":true,"status":"ok"}

HTTP Status: 200

--- POST /predict (valid image, digit 0) ---
{"confidence":0.9998,"predicted_digit":0,"probabilities":
{"0":0.9998,"1":0.0,"2":0.0,"3":0.0,"4":0.0,"5":0.0001,"6":0.0,"7":0.0,"8":0.0,"9":0.0}}

HTTP Status: 200

--- POST /predict/batch (valid batch) ---
{"count":1,"results":[{"confidence":0.9998,"predicted_digit":0,"probabilities":
{"0":0.9998,"1":0.0,"2":0.0,"3":0.0,"4":0.0,"5":0.0001,"6":0.0,"7":0.0,"8":0.0,"9":0.0}}]}

HTTP Status: 200

=== SECTION 2: Negative Tests (Error Handling) ===

--- POST /predict (missing 'image' field -> expect 400) ---
{"error":"Missing 'image' field in request body."}

HTTP Status: 400

--- POST /predict (wrong-size array -> expect 400) ---
{"error":"Expected an 8x8 (64-value) grayscale image, got 3 values."}

HTTP Status: 400

--- POST /predict (invalid JSON body -> expect 400) ---
{"error":"Request body must be valid JSON."}

HTTP Status: 400

--- POST /predict/batch (empty list -> expect 400) ---
{"error":"'images' must be a non-empty list of 64-value images."}

HTTP Status: 400

--- GET /nonexistent (unknown endpoint -> expect 404) ---
{"error":"Endpoint not found. See GET / for a list of available endpoints."}

HTTP Status: 404

--- GET /predict (wrong HTTP method -> expect 405) ---
{"error":"Method not allowed for this endpoint."}

HTTP Status: 405
```

7. Results Summary

#	Test	Method & Endpoint	Expected	Result
1	API information	GET /	200, JSON with name/endpoints	PASS
2	Health check	GET /health	200, model_loaded: true	PASS
3	Valid single prediction	POST /predict	200, correct digit predicted	PASS
4	Valid batch prediction	POST /predict/batch	200, all 3 digits correct	PASS
5	Missing 'image' field	POST /predict	400, structured error	PASS
6	Wrong-size image array	POST /predict	400, structured error	PASS
7	Invalid JSON body	POST /predict	400, structured error	PASS
8	Out-of-range pixel values	POST /predict	400, structured error	PASS
9	Empty batch list	POST /predict/batch	400, structured error	PASS
10	Mixed valid/invalid batch	POST /predict/batch	200, per-item error isolation	PASS
11	Unknown endpoint	GET /nonexistent	404, structured error	PASS
12	Wrong HTTP method	GET /predict	405, structured error	PASS

Total: 20 individual assertions across 12 test cases — 20/20 passed (100.0%).

8. Observations

- Every endpoint returns the correct HTTP status code for both valid and invalid input, and every response body is valid JSON in every case tested - including all negative/error cases, which is not guaranteed by default in many web frameworks (some return HTML error pages on unhandled exceptions).
- The batch endpoint correctly isolates a single malformed image's error without failing the other, valid images in the same request - verified directly by the mixed valid/invalid batch test (test case 10).
- No test produced a 500 Internal Server Error, meaning every anticipated failure mode is caught by explicit input validation rather than falling through to the generic exception handler.
- The curl-based results (Section 6.2) and the automated Python test results (Section 6.1) agree exactly, confirming the behavior is consistent regardless of which client tool issues the request.
- Response times for single predictions were consistently fast, since the model is loaded once at API startup rather than reloaded per request.

9. Deliverables Produced

Deliverable	File
Postman Collection	postman_collection.json
API testing report	API_TESTING_REPORT.md; also this PDF
Response logs and screenshots	Section 6 of this report; also test_run_log.txt, curl_output_log.txt
Automated test runner	test_api_validation.py
curl test script	curl_commands.sh
Interactive demonstration notebook	Task5_API_Testing.ipynb

10. Conclusion

The Task 4 Digit Classification API was systematically tested using a Postman Collection, equivalent curl commands, and an automated Python test runner, covering both functional correctness and error handling. All 20 assertions across 12 test cases passed (100%), confirming the API returns correct predictions for valid input and clean, structured error responses with appropriate HTTP status codes for every invalid input scenario tested.